

Functional Programming (FP) — Functional JavaScript শেখার মূল উদ্দেশ্য

JavaScript-এ **Functional Programming** অত্যন্ত গুরুত্বপূর্ণ—বিশেষ করে তুমি যখন **React**, **Redux**, **Node.js** এবং **Modern Full-Stack** ইকোসিস্টেমে কাজ করবে। এই চ্যাপ্টারে আমরা শুধুমাত্র `map()` বা `filter()` নয়, বরং Functional JavaScript-এর গভীরতম ভিত্তিগুলো—Pure Functions, Immutability, Composition, Currying এবং Declarative Data Pipelines—নিখুঁতভাবে শিখব।

Core Formula: Input Data → Pure Function (No Side Effect) → New Immutable Output Data

Part 01 — Functional Programming Fundamentals & Pure Functions (25.1 – 25.9)

25.1 Functional Programming কী?

Functional Programming (FP) হলো এমন একটি programming style যেখানে আমরা program-কে ছোট ছোট **function** দিয়ে তৈরি করি এবং function-কে data process করার প্রধান মাধ্যম হিসেবে ব্যবহার করি।

◆ সহজ ভাষায়

ধরো একটি restaurant-এ: > Raw ingredients → রান্নার function → Final food
JavaScript-এ: > Input data → Function → Output data

JAVASCRIPT FP

JAVASCRIPT

```
function double(number) {  
  return number * 2;  
}
```

```
console.log(double(5));
```

▶ CONSOLE OUTPUT

OUTPUT

10

এখানে:



FUNCTIONAL ARCHITECTURE

5 → double() → 10

অর্থাৎ function input নিয়ে output তৈরি করেছে।

25.2 Functional Programming-এর মূল বৈশিষ্ট্য

Functional JavaScript-এর কয়েকটি গুরুত্বপূর্ণ principle: 1. **Pure Functions** 2. **Immutability** 3. **First-Class Functions** 4. **Higher-Order Functions** 5. **Function Composition** 6. **Declarative Programming** 7. **Avoiding Side Effects** 8. **Data Transformation** 9. **Referential Transparency** এগুলো একে একে বুঝি।

25.3 Pure Function

একটি function তখন **Pure Function**, যখন: 1. একই input দিলে সবসময় একই output দেয় 2. বাইরের কোনো data পরিবর্তন করে না

◆ Example

JAVASCRIPT FP

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(10, 20));  
console.log(add(10, 20));
```

► CONSOLE OUTPUT

OUTPUT

30
30

একই input:

► CONSOLE OUTPUT

OUTPUT

10, 20

দিলে সবসময়:

► CONSOLE OUTPUT

OUTPUT

30

তাই এটি pure function।

25.4 Impure Function

যে function বাইরের state/data পরিবর্তন করে অথবা বাইরের changing value-এর ওপর নির্ভর করে তাকে সাধারণভাবে **Impure Function** বলা হয়।

JAVASCRIPT FP

JAVASCRIPT

```
let total = 0;  
  
function addToTotal(value) {  
  total += value;  
}  
  
addToTotal(100);  
  
console.log(total);
```

► CONSOLE OUTPUT

OUTPUT

100

Function-এর ভিতর থেকে বাইরের variable:

JAVASCRIPT FP

JAVASCRIPT

total;

পরিবর্তন হয়েছে। তাই এটি pure function নয়।

25.5 Pure vs Impure

Pure Function	Impure Function
একই input → একই output	output/state পরিবর্তিত হতে পারে
বাইরের data পরিবর্তন করে না	বাইরের data পরিবর্তন করতে পারে
Predictable	Less predictable
Testing সহজ	Testing তুলনামূলক কঠিন

♦ Pure

JAVASCRIPT FP

JAVASCRIPT

```
function square(x) {  
  return x * x;  
}
```

♦ Impure

JAVASCRIPT FP

JAVASCRIPT

```
let count = 0;  
  
function increase() {  
  count++;  
}
```

25.6 Real-Life Example

ধরো calculator:

► CONSOLE OUTPUT

OUTPUT

```
Input: 10 + 20  
Output: 30
```

Calculator-এর function:

JAVASCRIPT FP

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

এটি pure। কিন্তু যদি function:

JAVASCRIPT FP

JAVASCRIPT

```
function add(a, b) {  
  console.log("Calculating...");  
  return a + b;  
}
```

তাহলে function একটি **side effect** করছে—console-এ কিছু লিখছে।

25.7 Side Effect কী?

Function-এর বাইরে কোনো observable পরিবর্তন হলে সেটি **Side Effect** হতে পারে। উদাহরণ:

JAVASCRIPT FP

JAVASCRIPT

```
console.log();
```

JAVASCRIPT FP

JAVASCRIPT

```
document.body.innerHTML = "Hello";
```

JAVASCRIPT FP

JAVASCRIPT

```
localStorage.setItem("name", "Ripon");
```

JAVASCRIPT FP

JAVASCRIPT

```
database.save(user);
```

JAVASCRIPT FP

JAVASCRIPT

```
fetch("/api/users");
```

এসব operation সাধারণত side effect তৈরি করে।

25.8 Side Effect Example

JAVASCRIPT FP

JAVASCRIPT

```
let users = [];  
  
function addUser(user) {  
  users.push(user);  
}
```

এখানে:

JAVASCRIPT FP

JAVASCRIPT

```
users.push(user);
```

বাইরের `users` array পরিবর্তন করছে। এটি side effect।

25.9 Pure Version

আমরা নতুন array return করতে পারি:

JAVASCRIPT FP

JAVASCRIPT

```
let users = [];  
  
function addUser(users, user) {  
  return [...users, user];  
}  
  
const updatedUsers = addUser(users, "Ripon");  
  
console.log(users);  
console.log(updatedUsers);
```

► CONSOLE OUTPUT

OUTPUT

```
[]  
["Ripon"]
```

Original array পরিবর্তন হয়নি। এটাই Functional Programming-এর গুরুত্বপূর্ণ ধারণা।

25.10 Immutability

Immutability মানে হলো existing data সরাসরি পরিবর্তন না করা। ধরো:

JAVASCRIPT FP

JAVASCRIPT

```
const user = {
  name: "Ripon",
  age: 23,
};
```

এখন:

JAVASCRIPT FP

JAVASCRIPT

```
user.age = 24;
```

এখানে object-এর existing data পরিবর্তন হয়েছে। Functional style-এ আমরা নতুন object তৈরি করতে পারি।

JAVASCRIPT FP

JAVASCRIPT

```
const user = {
  name: "Ripon",
  age: 23,
};

const updatedUser = {
  ...user,
  age: 24,
};

console.log(user);
console.log(updatedUser);
```

▶ CONSOLE OUTPUT

OUTPUT

```
{ name: "Ripon", age: 23 }
{ name: "Ripon", age: 24 }
```

Original object unchanged!

25.11 Array Immutability

ধরো:

JAVASCRIPT FP

JAVASCRIPT

```
const numbers = [1, 2, 3];
```

এভাবে করলে:

JAVASCRIPT FP

JAVASCRIPT

```
numbers.push(4);
```

original array পরিবর্তিত হচ্ছে। Functional style:

JAVASCRIPT FP

JAVASCRIPT

```
const numbers = [1, 2, 3];

const newNumbers = [...numbers, 4];

console.log(numbers);
console.log(newNumbers);
```

▶ CONSOLE OUTPUT

OUTPUT

```
[1, 2, 3]
[1, 2, 3, 4]
```

25.12 First-Class Functions

Chapter 6-এ আমরা এটি দেখেছিলাম। JavaScript-এ function হলো **First-Class Citizen**। অর্থাৎ function:

- variable-এ রাখা যায়
- argument হিসেবে পাঠানো যায়
- return করা যায়
- object/property হিসেবে রাখা যায়
- array-এর মধ্যে রাখা যায়

♦ Example

JAVASCRIPT FP	JAVASCRIPT
<pre>function greet() { console.log("Hello!"); } const myFunction = greet; myFunction();</pre>	
▶ CONSOLE OUTPUT	OUTPUT
Hello!	

25.13 Function as a Value

JAVASCRIPT FP	JAVASCRIPT
<pre>const add = function (a, b) { return a + b; }; console.log(add(5, 10));</pre>	

এখানে:

JAVASCRIPT FP	JAVASCRIPT
add;	

একটি variable এবং এর value একটি function।

25.14 Function as Argument

JAVASCRIPT FP	JAVASCRIPT
<pre>function process(number, operation) { return operation(number); } function double(x) { return x * 2; } console.log(process(5, double));</pre>	
▶ CONSOLE OUTPUT	OUTPUT
10	

এখানে:

FUNCTIONAL ARCHITECTURE
<pre>5 ↓ process() ↓ double() ↓ 10</pre>

25.15 Higher-Order Function

যে function:

- অন্য function গ্রহণ করে

অথবা

- অন্য function return করে

তাকে **Higher-Order Function** বলে।

♦ Example

```
JAVASCRIPT FP JAVASCRIPT
function calculate(a, b, operation) {
  return operation(a, b);
}

function add(a, b) {
  return a + b;
}

console.log(calculate(10, 20, add));
```

```
► CONSOLE OUTPUT OUTPUT
30
```

এখানে:

```
JAVASCRIPT FP JAVASCRIPT
calculate();
```

একটি Higher-Order Function।

25.16 Built-in Higher-Order Functions

JavaScript-এর অনেক array method Higher-Order Function। সবচেয়ে গুরুত্বপূর্ণ:

```
► CONSOLE OUTPUT OUTPUT
map()
filter()
reduce()
forEach()
find()
some()
every()
sort()
```

এখন এগুলো Functional Programming perspective থেকে দেখি।

Part 03 — Core Array Transformations: map, filter & reduce (25.17 – 25.24)

25.17 map()

map() প্রতিটি element-এর ওপর function চালিয়ে **নতুন array** তৈরি করে।

```
JAVASCRIPT FP JAVASCRIPT
const numbers = [1, 2, 3, 4];

const doubled = numbers.map((number) => number * 2);

console.log(doubled);
```

```
► CONSOLE OUTPUT OUTPUT
[2, 4, 6, 8]
```

Original:

```
► CONSOLE OUTPUT OUTPUT
[1, 2, 3, 4]
```

পরিবর্তিত হয়নি।

25.18 Real-Life map()

ধরো student marks:

JAVASCRIPT FP

JAVASCRIPT

```
const marks = [50, 60, 70, 80];

const increasedMarks = marks.map((mark) => mark + 5);

console.log(increasedMarks);
```

► CONSOLE OUTPUT

OUTPUT

```
[55, 65, 75, 85]
```

25.19 filter()

filter() condition অনুযায়ী element নির্বাচন করে নতুন array দেয়।

JAVASCRIPT FP

JAVASCRIPT

```
const numbers = [10, 15, 20, 25, 30];

const evenNumbers = numbers.filter((number) => number % 2 === 0);

console.log(evenNumbers);
```

► CONSOLE OUTPUT

OUTPUT

```
[10, 20, 30]
```

25.20 Real-Life filter()

JAVASCRIPT FP

JAVASCRIPT

```
const students = [
  { name: "A", marks: 85 },
  { name: "B", marks: 45 },
  { name: "C", marks: 75 },
];

const passedStudents = students.filter((student) => student.marks >= 50);

console.log(passedStudents);
```

► CONSOLE OUTPUT

OUTPUT

```
[
  { name: "A", marks: 85 },
  { name: "C", marks: 75 }
]
```

25.21 reduce()

`reduce()` multiple valuesকে একটি final value-তে পরিণত করতে পারে।

```
JAVASCRIPT FP                                     JAVASCRIPT
const numbers = [10, 20, 30, 40];

const total = numbers.reduce((sum, number) => sum + number, 0);

console.log(total);
```

► CONSOLE OUTPUT

OUTPUT

100

Mental model:

🔥 FUNCTIONAL ARCHITECTURE

```
0
↓
0 + 10 = 10
↓
10 + 20 = 30
↓
30 + 30 = 60
↓
60 + 40 = 100
```

25.22 map() + filter() + reduce()

Functional JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ patternগুলোর একটি।

```
JAVASCRIPT FP                                     JAVASCRIPT
const prices = [100, 200, 300, 400];

const result = prices
  .filter((price) => price >= 200)
  .map((price) => price * 0.9)
  .reduce((total, price) => total + price, 0);

console.log(result);
```

♦ Step-by-step

প্রথম:

► CONSOLE OUTPUT

OUTPUT

[100, 200, 300, 400]

`filter()`:

► CONSOLE OUTPUT

OUTPUT

[200, 300, 400]

`map()`:

► CONSOLE OUTPUT

OUTPUT

[180, 270, 360]

`reduce()`:

► CONSOLE OUTPUT

OUTPUT

810

► CONSOLE OUTPUT

OUTPUT

810

এটি Functional Programming-এর খুব গুরুত্বপূর্ণ style।

25.23 Declarative Programming

Functional Programming সাধারণত **Declarative** style ব্যবহার করে।

♦ Imperative

মানো কীভাবে কাজ করতে হবে সেটা বলে দেওয়া।

JAVASCRIPT FP

JAVASCRIPT

```
const numbers = [1, 2, 3, 4];
const result = [];

for (let i = 0; i < numbers.length; i++) {
  result.push(numbers[i] * 2);
}

console.log(result);
```

♦ Declarative

কী result চাই সেটা প্রকাশ করা:

JAVASCRIPT FP

JAVASCRIPT

```
const numbers = [1, 2, 3, 4];

const result = numbers.map((number) => number * 2);

console.log(result);
```

দুটোর output একই:

► CONSOLE OUTPUT

OUTPUT

[2, 4, 6, 8]

25.24 Imperative vs Declarative

Imperative	Declarative
কীভাবে করতে হবে	কী চাই
বেশি control	বেশি abstraction
loops বেশি	methods বেশি
step-by-step	transformation-focused

25.25 Function Composition

একাধিক ছোট function একসাথে ব্যবহার করে বড় কাজ তৈরি করাকে **Function Composition** বলা হয়। ধরো:



```
Input
↓
double
↓
add10
↓
square
↓
Output
```

Code:

JAVASCRIPT FP

JAVASCRIPT

```
const double = (x) => x * 2;

const add10 = (x) => x + 10;

const square = (x) => x * x;

const result = square(add10(double(5)));

console.log(result);
```

◆ Calculation

▶ CONSOLE OUTPUT

OUTPUT

```
double(5)
= 10

add10(10)
= 20

square(20)
= 400
```

▶ CONSOLE OUTPUT

OUTPUT

```
400
```

25.26 নিজের compose() Function

আমরা composition-এর জন্য function তৈরি করতে পারি।

JAVASCRIPT FP

JAVASCRIPT

```
const compose = (f, g) => (value) => f(g(value));

const double = (x) => x * 2;
const add10 = (x) => x + 10;

const process = compose(add10, double);

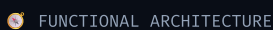
console.log(process(5));
```

▶ CONSOLE OUTPUT

OUTPUT

```
20
```

কাঁকড়া:



```
5
↓
double → 10
↓
add10 → 20
```

25.27 pipe()

`compose()` সাধারণত right-to-left কাজ করতে পারে। আর `pipe()` সাধারণত left-to-right।

JAVASCRIPT FP

JAVASCRIPT

```
const pipe =
  (... functions) =>
  (value) =>
    functions.reduce((result, fn) => fn(result), value);
```

এখন:

JAVASCRIPT FP

JAVASCRIPT

```
const double = (x) => x * 2;
const add10 = (x) => x + 10;
const square = (x) => x * x;

const process = pipe(double, add10, square);

console.log(process(5));
```

► CONSOLE OUTPUT

OUTPUT

400

কারণ:

🕒 FUNCTIONAL ARCHITECTURE

```
5
↓
double → 10
↓
add10 → 20
↓
square → 400
```

25.28 Immutability with Objects

ধরো:

JAVASCRIPT FP

JAVASCRIPT

```
const user = {
  name: "Ripon",
  skills: ["HTML", "CSS"],
};
```

আমরা সরাসরি:

JAVASCRIPT FP

JAVASCRIPT

```
user.skills.push("JavaScript");
```

না করে নতুন object তৈরি করতে পারি:

JAVASCRIPT FP

JAVASCRIPT

```
const updatedUser = {
  ... user,
  skills: [... user.skills, "JavaScript"],
};

console.log(updatedUser);
```

► CONSOLE OUTPUT

OUTPUT

```
{
  name: "Ripon",
  skills: ["HTML", "CSS", "JavaScript"]
}
```

25.29 Updating Array Object

JAVASCRIPT FP

JAVASCRIPT

```
const users = [
  { id: 1, name: "A", active: true },
  { id: 2, name: "B", active: false },
];
```

User 2-কে active করতে:

JAVASCRIPT FP

JAVASCRIPT

```
const updatedUsers = users.map((user) =>
  user.id === 2 ? { ...user, active: true } : user,
);

console.log(updatedUsers);
```

► CONSOLE OUTPUT

OUTPUT

```
[
  { id: 1, name: "A", active: true },
  { id: 2, name: "B", active: true }
]
```

এটি React-এ খুবই গুরুত্বপূর্ণ pattern।

25.30 Why React Developers Need This

React state সরাসরি mutate করা উচিত নয়। খারাপ pattern:

JAVASCRIPT FP

JAVASCRIPT

```
user.name = "New Name";
```

Better:

JAVASCRIPT FP

JAVASCRIPT

```
setUser({
  ...user,
  name: "New Name",
});
```

Array-এর ক্ষেত্রে:

JAVASCRIPT FP

JAVASCRIPT

```
setUsers(
  users.map((user) => (user.id === 2 ? { ...user, active: true } : user)),
);
```

এই কারণেই Functional JavaScript ভালোভাবে জানা React-এর জন্য খুব গুরুত্বপূর্ণ।

25.31 Referential Transparency

একটি expression যদি তার result দিয়ে safely replace করা যায় এবং program-এর behavior পরিবর্তন না হয়, তাকে **Referential Transparency** বলা হয়। ধরো:

JAVASCRIPT FP

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

তাহলে:

JAVASCRIPT FP

JAVASCRIPT

```
add(10, 20);
```

সবসময়:

► CONSOLE OUTPUT

OUTPUT

```
30
```

তাই:

JAVASCRIPT FP

JAVASCRIPT

```
const x = add(10, 20);
```

কে:

JAVASCRIPT FP

JAVASCRIPT

```
const x = 30;
```

দিয়ে replace করা যায়।

25.32 Currying

Currying হলো একটি function-এর multiple arguments-কে একসাথে নেওয়ার পরিবর্তে একবারে একটি করে নেওয়া। Normal:

JAVASCRIPT FP

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(10, 20));
```

Curried:

JAVASCRIPT FP

JAVASCRIPT

```
function add(a) {  
  return function (b) {  
    return a + b;  
  };  
}  
  
console.log(add(10)(20));
```

► CONSOLE OUTPUT

OUTPUT

```
30
```

Arrow version:

JAVASCRIPT FP

JAVASCRIPT

```
const add = (a) => (b) => a + b;  
  
console.log(add(10)(20));
```

25.33 Currying Real-Life Example

ধরো tax calculation:

```
JAVASCRIPT FP JAVASCRIPT
const addTax = (taxRate) => (price) => price + price * taxRate;

const addVAT = addTax(0.15);

console.log(addVAT(1000));
console.log(addVAT(2000));
```

► CONSOLE OUTPUT

OUTPUT

```
1150
2300
```

এখানে:

```
JAVASCRIPT FP JAVASCRIPT
addTax(0.15);
```

একটি reusable function তৈরি করেছে।

25.34 Partial Application

Currying-এর সঙ্গে সম্পর্কিত আরেকটি concept হলো **Partial Application**। ধরো:

```
JAVASCRIPT FP JAVASCRIPT
function multiply(a, b, c) {
  return a * b * c;
}
```

আমরা কিছু argument আগে সেট করতে পারি।

```
JAVASCRIPT FP JAVASCRIPT
function multiplyBy(a) {
  return (b, c) => a * b * c;
}

const multiplyBy2 = multiplyBy(2);

console.log(multiplyBy2(3, 4));
```

► CONSOLE OUTPUT

OUTPUT

```
24
```

25.35 Closure + Functional Programming

Currying এবং functional programming-এ closure অনেক গুরুত্বপূর্ণ।

```
JAVASCRIPT FP JAVASCRIPT
function createMultiplier(multiplier) {
  return function (number) {
    return number * multiplier;
  };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(10));
console.log(triple(10));
```

► CONSOLE OUTPUT

OUTPUT

```
20
30
```

Inner function `multiplier` variable-টিকে মনে রাখছে। এটাই closure।

25.36 Avoiding Mutation

Functional Programming-এর প্রধান শর্ত হলো বিদ্যমান ডেটা সরাসরি পরিবর্তন (mutate) না করা।

❌ ভুল পদ্ধতি: MUTATION (SIDE EFFECT)

❌ MUTATION / IMPURE

```
const numbers = [1, 2, 3];
numbers.push(4); // ❌ Original array সরাসরি mutate হয়ে গেছে!
```

সতর্কতা: push() মেথড মূল অ্যারে পরিবর্তন করে ফেলে, যার ফলে অ্যাপ্লিকেশনের স্টেট অপ্রত্যাশিতভাবে বদলে যেতে পারে।

সঠিক সমাধান (Immutable Style): Spread operator (...) ব্যবহার করে নতুন কপি তৈরি করুন।

✅ সঠিক পদ্ধতি: IMMUTABILITY (PURE PATTERN)

✅ PURE / IMMUTABLE

```
const numbers = [1, 2, 3];
const newNumbers = [...numbers, 4]; // ✅ মূল array অক্ষত রেখে নতুন array রিটার্ন
```

25.37 Sorting Without Mutation

একটি বহুল পরিচিত ট্রিকি বিষয় হলো জাভাস্ক্রিপ্টের বিল্ট-ইন sort() মেথড মূল অ্যারে মিউটেট করে।

❌ MUTATING SORT: মূল অ্যারে নষ্ট করে

❌ MUTATION / IMPURE

```
const numbers = [3, 1, 2];
const sorted = numbers.sort(); // ❌ মূল numbers অ্যারেও sort হয়ে যাবে!
```

Functional সমাধান ১: স্প্রেড অপারেটর দিয়ে কপি বানিয়ে সর্ট করা:

✅ SAFE PATTERN: [...NUMBERS].SORT()

✅ PURE / IMMUTABLE

```
const numbers = [3, 1, 2];
const sorted = [...numbers].sort((a, b) => a - b); // ✅ Spread কপি করে sort
console.log(numbers); // [3, 1, 2]
console.log(sorted); // [1, 2, 3]
```

Functional সমাধান ২ (ES2023+ Standard): আধুনিক toSorted() মেথড ব্যবহার করা:

✅ MODERN PATTERN: TOSORTED()

✅ PURE / IMMUTABLE

```
// Modern ES2023+ Native Immutable Method:
const sortedNative = numbers.toSorted((a, b) => a - b); // ✅ মূল অ্যারে অক্ষত থাকে
```

25.38 forEach() vs map()

♦ forEach()

সাধারণত side effect করার জন্য:

JAVASCRIPT FP

JAVASCRIPT

```
const numbers = [1, 2, 3];

numbers.forEach((number) => {
  console.log(number);
});
```

♦ map()

নতুন transformed array তৈরি করার জন্য:

JAVASCRIPT FP

JAVASCRIPT

```
const doubled = numbers.map((number) => number * 2);
```

♦ মনে রাখবে

🕒 FUNCTIONAL ARCHITECTURE

forEach → কাজ করে
map → নতুন data তৈরি করে

25.39 find()

প্রথম matching element return করে।

JAVASCRIPT FP

JAVASCRIPT

```
const users = [
  { id: 1, name: "A" },
  { id: 2, name: "B" },
];

const user = users.find((user) => user.id === 2);

console.log(user);
```

► CONSOLE OUTPUT

OUTPUT

```
{ id: 2, name: "B" }
```

25.40 some()

কমপক্ষে একটি element condition satisfy করলে true।

JAVASCRIPT FP

JAVASCRIPT

```
const marks = [40, 55, 70];

const hasPassed = marks.some((mark) => mark >= 50);

console.log(hasPassed);
```

► CONSOLE OUTPUT

OUTPUT

```
true
```

25.41 every()

সব element condition satisfy করলে true।

JAVASCRIPT FP

JAVASCRIPT

```
const marks = [60, 70, 80];

const allPassed = marks.every((mark) => mark >= 50);

console.log(allPassed);
```

► CONSOLE OUTPUT

OUTPUT

```
true
```

25.42 flatMap()

map() করার পরে nested array flatten করার জন্য ব্যবহার করা যায়।

JAVASCRIPT FP

JAVASCRIPT

```
const students = [
  {
    name: "A",
    skills: ["HTML", "CSS"],
  },
  {
    name: "B",
    skills: ["JS", "React"],
  },
];

const skills = students.flatMap((student) => student.skills);

console.log(skills);
```

► CONSOLE OUTPUT

OUTPUT

```
["HTML", "CSS", "JS", "React"]
```

25.43 Functional Data Pipeline

এখন একটি realistic example দেখি। ধরো e-commerce products:

JAVASCRIPT FP

JAVASCRIPT

```
const products = [
  { name: "Laptop", price: 80000, stock: 5 },
  { name: "Mouse", price: 1000, stock: 20 },
  { name: "Keyboard", price: 3000, stock: 0 },
  { name: "Monitor", price: 25000, stock: 8 },
];
```

আমরা available এবং 5000-এর বেশি price-এর product চাই।

JAVASCRIPT FP

JAVASCRIPT

```
const result = products
  .filter((product) => product.stock > 0)
  .filter((product) => product.price > 5000)
  .map((product) => ({
    ...product,
    price: product.price * 0.9,
  }));

console.log(result);
```

► CONSOLE OUTPUT

OUTPUT

```
[
  { name: "Laptop", price: 72000, stock: 5 },
  { name: "Monitor", price: 22500, stock: 8 }
]
```

এখানে:

🔗 FUNCTIONAL ARCHITECTURE

```
products
  ↓
filter stock
  ↓
filter price
  ↓
map discount
  ↓
result
```

এটাকে **Data Transformation Pipeline** হিসেবে ভাবতে পারো।

25.44 Functional Programming + API

Backend/API থেকে data আসলে:

JAVASCRIPT FP

JAVASCRIPT

```
const users = [
  { name: "A", active: true },
  { name: "B", active: false },
  { name: "C", active: true },
];
```

শুধু active users-এর names:

JAVASCRIPT FP

JAVASCRIPT

```
const activeNames = users
  .filter((user) => user.active)
  .map((user) => user.name);

console.log(activeNames);
```

► CONSOLE OUTPUT

OUTPUT

```
["A", "C"]
```

এটি React/Node project-এ প্রতিদিনের মতো ব্যবহার হবে।

25.45 Functional Programming-এর সুবিধা

♦ 1. Code predictable

Pure function-এর behavior সহজে বোঝা যায়।

♦ 2. Testing সহজ

JAVASCRIPT FP

JAVASCRIPT

```
add(10, 20);
```

এর expected result:

► CONSOLE OUTPUT

OUTPUT

```
30
```

সহজে test করা যায়।

♦ 3. Reusable

ছোট ছোট function বারবার ব্যবহার করা যায়।

♦ 4. Maintainable

বড় code ছোট function-এ ভাগ করা যায়।

♦ 5. Debugging সহজ

Pipeline-এর প্রতিটি step আলাদা করে পরীক্ষা করা যায়।

♦ 6. React-এর জন্য গুরুত্বপূর্ণ

State transformation-এ immutable patterns খুব গুরুত্বপূর্ণ।

25.46 Functional Programming-এর অসুবিধা

সবসময় functional style ব্যবহার করাই সঠিক নয়। কিছু ক্ষেত্রে:

- বেশি function তৈরি হতে পারে
- অতিরিক্ত abstraction code কঠিন করতে পারে
- নতুনদের কাছে chaining confusing হতে পারে
- performance-sensitive জায়গায় unnecessary transformation হতে পারে

তাই লক্ষ্য হবে: > **Functional style যেখানে code পরিষ্কার ও predictable হয়, সেখানে ব্যবহার করা।**

25.47 Real Project Example — Student Result

JAVASCRIPT FP

JAVASCRIPT

```
const students = [  
  { name: "Rahim", marks: 80 },  
  { name: "Karim", marks: 45 },  
  { name: "Nadia", marks: 90 },  
  { name: "Sadia", marks: 35 },  
];
```

Passed students:

JAVASCRIPT FP

JAVASCRIPT

```
const passedStudents = students  
  .filter((student) => student.marks >= 40)  
  .map((student) => ({  
    ...student,  
    status: "Passed",  
  }));  
  
console.log(passedStudents);
```

► CONSOLE OUTPUT

OUTPUT

```
[  
  { name: "Rahim", marks: 80, status: "Passed" },  
  { name: "Karim", marks: 45, status: "Passed" },  
  { name: "Nadia", marks: 90, status: "Passed" }  
]
```

25.48 Result Calculation with `reduce()`

JAVASCRIPT FP

JAVASCRIPT

```
const students = [
  { name: "Rahim", marks: 80 },
  { name: "Karim", marks: 45 },
  { name: "Nadia", marks: 90 },
];

const totalMarks = students.reduce(
  (total, student) => total + student.marks,
  0,
);

const average = totalMarks / students.length;

console.log("Total:", totalMarks);
console.log("Average:", average);
```

► CONSOLE OUTPUT

OUTPUT

```
Total: 215
Average: 71.66666666666667
```

25.49 Functional Programming Mental Model

এই flow-টা মনে রাখো:

🕒 FUNCTIONAL ARCHITECTURE

```
DATA
↓
filter()
↓
map()
↓
reduce()
↓
RESULT
```

Example:

JAVASCRIPT FP

JAVASCRIPT

```
const result = products
  .filter((product) => product.stock > 0)
  .map((product) => product.price)
  .reduce((total, price) => total + price, 0);
```

অর্থাৎ:

🕒 FUNCTIONAL ARCHITECTURE

```
Products
↓
Available products
↓
Only prices
↓
Total price
```

25.50 🔥 Must Know

Chapter 25 থেকে অবশ্যই জানতে হবে:

♦ Basic

- Functional Programming
- Pure Function
- Impure Function
- Side Effect
- Immutability
- First-Class Function
- Higher-Order Function
- Declarative Programming

♦ Array Functional Methods

- `map()`
- `filter()`
- `reduce()`
- `find()`
- `some()`
- `every()`
- `flatMap()`

♦ Advanced

- Function Composition
- `pipe()`
- Currying
- Partial Application
- Closure + Functional Programming
- Referential Transparency
- Immutable object update
- Immutable array update
- Data transformation pipeline

25.Quick 🧠 Quick Cheat Sheet — Functional JavaScript Reference

Functional JavaScript এবং ডেটা রূপান্তরের দ্রুত রেফারেন্সের জন্য সম্পূর্ণ সিনট্যাক্স টেবিল:

Concept	Meaning
Pure Function	Same input → same output
Impure Function	External state/side effect থাকতে পারে
Side Effect	Function-এর বাইরে observable change
Immutability	Existing data সরাসরি পরিবর্তন না করা
First-Class Function	Function-কে value হিসেবে ব্যবহার করা
Higher-Order Function	Function নেয়/return করে
<code>map()</code>	Data transform
<code>filter()</code>	Data select
<code>reduce()</code>	Multiple values → one value
<code>find()</code>	First matching item
<code>some()</code>	অন্তত একটি match
<code>every()</code>	সব match
Composition	Multiple functions combine
Currying	Arguments একবারে একটি করে নেওয়া
Declarative	কী চাই সেটা বলা

Functional Programming-এর কনসেপ্টগুলোকে আত্মস্থ করার জন্য নিচে বাস্তবভিত্তিক ৫টি চ্যালেঞ্জের সম্পূর্ণ প্রোডাকশন সলিউশন দেওয়া হলো:

Practice 1: Double with map() — Immutable Numeric Transformation

Problem Requirement: `numbers = [2, 4, 6, 8]` অ্যারে থেকে `map()` ব্যবহার করে প্রতিটি সংখ্যাকে দ্বিগুণ করে `[4, 8, 12, 16]` নতুন অ্যারে তৈরি করুন।

PRACTICE 1 MODEL SOLUTION

✓ PURE / IMMUTABLE

```
const numbers = [2, 4, 6, 8];

// Pure transformation using map()
const doubled = numbers.map(num => num * 2);

console.log("Original:", numbers);
console.log("Doubled:", doubled);
```

▶ CONSOLE OUTPUT

OUTPUT

```
Original: [2, 4, 6, 8]
Doubled: [4, 8, 12, 16]
```

Practice 2: Filter Condition — Selecting Elements >= 20

Problem Requirement: `numbers = [10, 15, 20, 25, 30, 35]` থেকে `filter()` ব্যবহার করে শুধুমাত্র ২০ বা তার বেশি সংখ্যাগুলো ফিল্টার করে বের করুন।

PRACTICE 2 MODEL SOLUTION

✓ PURE / IMMUTABLE

```
const numbers = [10, 15, 20, 25, 30, 35];

// Declarative filtering with pure predicate
const filtered = numbers.filter(num => num >= 20);

console.log("Result:", filtered);
```

▶ CONSOLE OUTPUT

OUTPUT

```
Result: [20, 25, 30, 35]
```

Practice 3: Reduce Accumulation — Calculating Array Total

Problem Requirement: `numbers = [10, 20, 30, 40]` থেকে `reduce()` ব্যবহার করে মোট যোগফল 100 বের করুন।

PRACTICE 3 MODEL SOLUTION

✓ PURE / IMMUTABLE

```
const numbers = [10, 20, 30, 40];

// Sum accumulation with initial value 0
const total = numbers.reduce((accumulator, current) => accumulator + current, 0);

console.log("Total Sum:", total);
```

▶ CONSOLE OUTPUT

OUTPUT

```
Total Sum: 100
```

Practice 4: Passed Students Filter — Object Array Query

Problem Requirement: শিক্ষার্থীদের তালিকা থেকে `filter()` ব্যবহার করে শুধুমাত্র পাস করা শিক্ষার্থীদের (`marks >= 40`) আলাদা করুন।

PRACTICE 4 MODEL SOLUTION

✓ PURE / IMMUTABLE

```
const students = [
  { name: "A", marks: 80 },
  { name: "B", marks: 35 },
  { name: "C", marks: 70 },
  { name: "D", marks: 45 },
];

const passedStudents = students.filter(student => student.marks >= 40);

console.log("Passed Students:", passedStudents);
```

▶ CONSOLE OUTPUT

OUTPUT

```
Passed Students: [
  { name: 'A', marks: 80 },
  { name: 'C', marks: 70 },
  { name: 'D', marks: 45 }
]
```

Practice 5: Functional Pipeline — filter() + map() + reduce() Chain

Problem Requirement: পণ্য তালিকা থেকে: ১. স্টকে থাকা পণ্য নির্বাচন করুন (`stock > 0`), ২. তাদের মূল্য বের করুন, এবং ৩. মোট স্টক পণ্যের মূল্য বের করতে মেথড চেইনিং পাইপলাইন তৈরি করুন।

PRACTICE 5 MODEL SOLUTION

✓ PURE / IMMUTABLE

```
const products = [
  { name: "Laptop", price: 80000, stock: 5 },
  { name: "Mouse", price: 1000, stock: 0 },
  { name: "Monitor", price: 20000, stock: 3 },
];

// Clean functional data pipeline
const totalInStockPrice = products
  .filter(item => item.stock > 0)
  .map(item => item.price * item.stock)
  .reduce((sum, currentVal) => sum + currentVal, 0);

console.log("Total In-Stock Value: ₳", totalInStockPrice);
```

► CONSOLE OUTPUT

OUTPUT

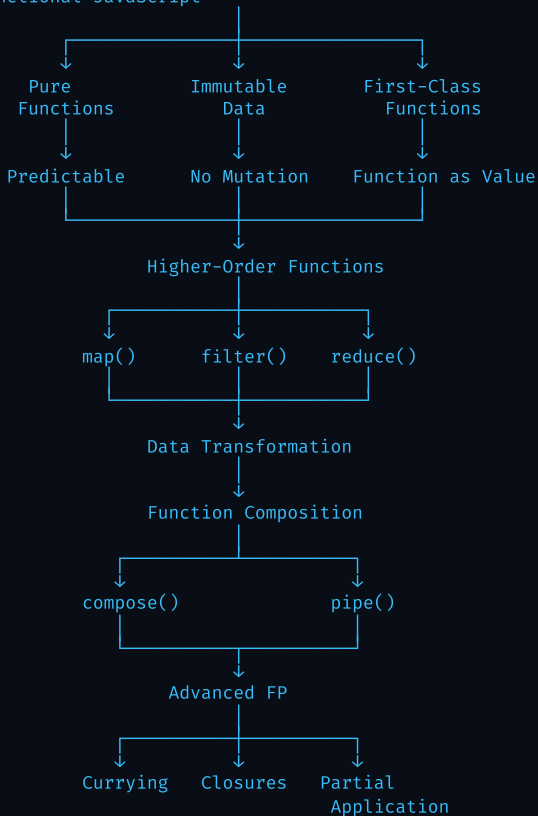
Total In-Stock Value: ₳ 460000

25.Map 🧠 Final Mental Map & Functional Ecosystem

JavaScript Functional Programming-এর সামগ্রিক আর্কিটেকচার এবং মেথড প্রবাহের চূড়ান্ত চিত্র:

FUNCTIONAL JAVASCRIPT MASTER ARCHITECTURE TREE

Functional JavaScript



★ এক নজরে Chapter 25 এর সারসংক্ষেপ

Key Takeaway: Functional JavaScript হলো ডেটাকে ছোট, রিইউজেবল ও প্রেডিক্টেবল ফাংশনের পাইপলাইনে রূপান্তর করার শিল্প—যেখানে Pure Functions, Immutability এবং map/filter/reduce-এর মতো হায়ার-অর্ডার ফাংশনগুলো কোডকে বাগ-মুক্ত, টেস্টেবল এবং সহজে মেইনটেইনেবল করে তোলে।